

AI1110 - Probability And Random Variables

Programming Assignment - April 2026

Instructions

- You must use **only** `np.random.rand()` as the source of randomness.
 - You are **not allowed** to use:
 - `np.random.normal`, `np.random.exponential`, `np.random.choice`, etc.
 - Use `numpy` and `matplotlib` for computation and plotting. Usage of other libraries will result in a penalty.
 - All plots must have labels, titles, and legends.
-

Question 1: Generating a Biased Coin from a Fair Coin

You are given access to a fair coin:

$$\mathbb{P}(H) = \frac{1}{2}, \quad \mathbb{P}(T) = \frac{1}{2}.$$

- 1.1) Using only `np.random.rand()`, implement a function `fair_coin()` that simulates a fair coin toss and returns:
 - **True** (heads) with probability $1/2$,
 - **False** (tails) with probability $1/2$.
- 1.2) Let $p \in (0, 1)$ be given. Using only repeated calls to `fair_coin()`, design an algorithm to generate a Bernoulli random variable X such that

$$\mathbb{P}(X = 1) = p, \quad \mathbb{P}(X = 0) = 1 - p.$$

- 1.3) Explain the idea behind your algorithm using the binary expansion of p . Clearly describe how successive fair coin tosses can be used to simulate the required biased outcome.
- 1.4) Implement a function `biased_coin(p)` based on your method in Step 1.2.
- 1.5) For the values $p = 0.3, 0.7, 0.125$, generate $n = 10^5$ samples using `biased_coin(p)` and compute the empirical probability of obtaining 1. Compare the empirical values with the theoretical values.
- 1.6) For each value of p , track the number of fair coin tosses used to generate each output. Compute and print the average number of fair coin tosses required per sample.

- 1.7) For one value of p (e.g., $p = 0.3$), plot the empirical Probability Mass Function (PMF) of the generated samples and verify that it matches the desired Bernoulli distribution.

Hint: Express p in binary form:

$$p = 0.b_1b_2b_3\dots$$

Generate a sequence of fair coin tosses and interpret heads/tails as binary digits. At each step, compare the generated binary sequence with the binary expansion of p . Use this comparison to decide whether to output 1 or 0.

Important: You must use only `np.random.rand()` as the source of randomness. You are not allowed to use any built-in sampling functions such as `numpy.random.choice`.

Question 2: Inverse CDF and Rejection Sampling

Consider the target density

$$f(x) = \frac{3}{2}(e^{-x} - e^{-3x}), \quad x \geq 0,$$

and the proposal density

$$g(x) = e^{-x}, \quad x \geq 0.$$

- 2.1) Find the smallest M such that $\frac{f(x)}{g(x)} \leq M$, $\forall x \geq 0$.
- 2.2) Consider the exponential distribution with density $g(x) = e^{-x}$, $x \geq 0$. Derive its cumulative distribution function (CDF) and obtain the inverse CDF explicitly.
- 2.3) Using only `np.random.rand()`, implement a function to generate samples from the proposal distribution $g(x)$ using the inverse CDF derived in Step 2.2.
- 2.4) Using the samples generated in Step 2.2, plot the empirical CDF and compare it with the theoretical CDF of the exponential distribution: $F(x) = 1 - e^{-x}$, $x \geq 0$.
- 2.5) Using the proposal samples from Step 2.2, implement the rejection sampling algorithm to generate samples from the target density $f(x)$. Your implementation must:
- generate a proposal sample Y ,
 - generate an independent uniform random variable $U \sim \text{Uniform}(0, 1)$,
 - accept Y if

$$U \leq \frac{f(Y)}{Mg(Y)},$$

- otherwise reject and repeat.

Continue this procedure until you obtain 10^5 accepted samples.

- 2.6) Using the accepted samples from Step 2.5, compute and print:
- the empirical mean,
 - the empirical variance,
 - the empirical acceptance rate (ratio of accepted samples to total generated proposals).
- 2.7) Show that the theoretical acceptance rate of the rejection sampling algorithm is $\frac{1}{M}$.

- 2.8) Plot the normalized histogram of the generated samples from Step 2.5 and superimpose the theoretical density $f(x)$ on the same plot. Clearly label axes and include a legend.
- 2.9) Show that $f(x)$ is a valid probability density function by verifying:
- non-negativity,
 - total integral equals 1.

Further, compute the theoretical mean $\mathbb{E}[X]$ of the distribution.

Instructions:

- You must generate samples in step 2.3 by applying **inverse CDF** transform on uniform samples generated using `np.random.rand()`. You are *not allowed* to use `np.random.exponential`, `np.random.choice`, etc.
- The empirical CDF of X is calculated by generating a large number of samples of X and plotting the cumulative proportion of the unique values taken by these samples.
- The quantities in step 2.1, 2.2, 2.5, 2.7 are to be found theoretically using the given PMF/PDF and not empirically.
- You have to write your own function for generating the empirical CDF. You should **NOT** use `matplotlib.pyplot.ecdf` or any other such built-in functions.

Question 3: Rare-Event probability estimation using Importance Sampling

Let $X \sim \mathcal{N}(0, 1)$. Estimate: $\mathbb{P}(X > 4)$

- Generate 10^6 standard Normal ($\mathcal{N}(0, 1)$) samples using the Box–Muller transform on uniform samples generated using `np.random.rand()`.
- Estimate the empirical probability $\mathbb{P}(X > 4)$ from the samples generated in step 3.1.
- Explain why this estimator is inefficient.
- Use importance sampling with the proposal distribution

$$Y \sim \mathcal{N}(4, 1).$$

Generate 10^6 fresh samples from $\mathcal{N}(0, 1)$ using Box-Mueller transformation of uniform samples and then shift the standard Normal samples to obtain samples from $\mathcal{N}(4, 1)$. That is, if $Z \sim \mathcal{N}(0, 1)$, then use the transformation

$$Y = Z + 4$$

to obtain $Y \sim \mathcal{N}(4, 1)$.

Note: You should not generate $\mathcal{N}(4, 1)$ samples directly using any built-in routine, but only by shifting Box–Muller generated $\mathcal{N}(0, 1)$ samples.

- Derive the likelihood ratio

$$w(y) = \frac{f_X(y)}{f_Y(y)},$$

where f_X is the density of $\mathcal{N}(0, 1)$ and f_Y is the density of $\mathcal{N}(4, 1)$.

- 3.6) Using the samples generated in Step 3.4, estimate $\mathbb{P}(X > 4)$ using importance sampling. More specifically, if $Y_1, Y_2, \dots, Y_n$ are the samples drawn from $\mathcal{N}(4, 1)$, compute

$$\hat{p}_{IS} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(Y_i > 4) w(Y_i),$$

where $\mathbf{1}(Y_i > 4)$ is 1 if $Y_i > 4$ and 0 otherwise. Print the numerical value of this estimate. Also print the average value of the importance weights over the rare-event region, i.e.,

$$\frac{\sum_{i=1}^n \mathbf{1}(Y_i > 4) w(Y_i)}{\sum_{i=1}^n \mathbf{1}(Y_i > 4)},$$

and compare the importance sampling estimate with the direct Monte Carlo estimate from Step 3.2.

- 3.7) Repeat Steps 3.2 and 3.6 independently 20 times each. In every repetition:

- generate a fresh set of 10^6 samples for the direct Monte Carlo estimator,
- generate a fresh set of 10^6 proposal samples for the importance sampling estimator,
- compute one estimate from each method.

Store the 20 direct Monte Carlo estimates in one array and the 20 importance sampling estimates in another array. Then compute and print:

- the sample mean of the 20 direct Monte Carlo estimates,
- the sample variance of the 20 direct Monte Carlo estimates,
- the sample mean of the 20 importance sampling estimates,
- the sample variance of the 20 importance sampling estimates.

Based on these values, comment on which method is more reliable for estimating the rare-event probability $\mathbb{P}(X > 4)$.

Question 4: CLT Verification Using Empirical CDF

- 4.1) Generate $n = 10^5$ samples according to a given distribution.

Example: (i) Exponential($\lambda = 1$), (ii) Uniform(0,2), (OR) (iii) a PMF on a finite support

$$p_X(k) = \begin{cases} 0.4, & \text{if } k = 1 \\ 0.3, & \text{if } k = 2 \\ 0.2, & \text{if } k = 3 \\ 0.1, & \text{if } k = 4 \\ 0, & \text{otherwise.} \end{cases}$$

- 4.2) For a specified value of N , say $N = 50$, generate the empirical CDF of the sample mean

$$S_N = \frac{\sum_{i=1}^N Y_i}{N},$$

where Y_i are sample values from step 4.1.

- 4.3) The central limit theorem asserts that the CDF of the sample mean converges to a Gaussian CDF with appropriate parameters. Find the mean and variance of the limiting Gaussian CDF for the given PMF.

- 4.4) Overlay the empirical CDF of the sample mean obtained in step 4.2 and the empirical CDF of a Gaussian random variable with parameters obtained in step 4.3 in the same plot.

Instructions:

- (a) You must generate samples in step 4.1 by applying transformations (for example: **inverse CDF**) on uniform samples generated using `np.random.rand()`. You are *not allowed* to use `np.random.normal`, `np.random.exponential`, `np.random.choice`, etc.
- (b) The empirical CDF of X is calculated by generating a large number of samples of X and plotting the cumulative proportion of the unique values taken by these samples.
- (c) The mean and variance in step 4.3 are to be found numerically using the given PMF/PDF and not empirically.
- (d) You have to write your own function for generating the empirical CDF. You should **NOT** use `matplotlib.pyplot.ecdf` or any other such built-in functions.
- (e) If you find the mean and variance in Step 4.3 to be μ and σ^2 , the Gaussian CDF in step 4.4 needs to be generated empirically by sampling from $\mathcal{N}(\mu, \sigma^2)$. You are *not allowed* to use `numpy.random.normal`. You must generate Gaussian random variables using the **Box–Muller transform** starting from `np.random.rand()`.
- (f) In step 4.4 above, the number of samples used in generating both the ECDFs should be the same. But this need not be the same as n or N in the given problem.

Question 5: Multi-variate Gaussian Random Variables

- 5.1) Generate $n = 10^5$ samples of the random vector $\mathbf{X} = [X_1, X_2, \dots, X_N]^T$, where each X_i is sampled from a standard normal distribution, i.e., $\mathcal{N}(0, 1)$.
- 5.2) You will be given an $N \times N$ matrix $\mathbf{A}$ and a $N \times 1$ real vector $\mathbf{b}$, using which you have to generate the sample vector $\mathbf{Y} = [Y_1, Y_2, \dots, Y_N]^T$ as $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$.
- 5.3) Generate the empirical CDF of Y_i , for a given index i , plot it and compare with the appropriate Gaussian CDF (the parameters of the Gaussian empirical CDF should be derived from $\mathbf{A}$ and $\mathbf{b}$).
- 5.4) Compute the empirical covariance matrix $\mathbf{K}_{\text{emp}}$ of $\mathbf{Y}$. Also, compute the actual covariance matrix $\mathbf{K}_{\text{actual}}$ of $\mathbf{Y}$ (using $\mathbf{A}$ and $\mathbf{b}$).
- 5.5) Find the mean squared error between $\mathbf{K}_{\text{emp}}$ and $\mathbf{K}_{\text{actual}}$ and plot it as a function of the number of samples n in step 5.1, for $n \in \{10, 100, 1000, 10^4, 10^5, 10^6\}$.

Instructions:

- (a) You must generate samples in step 5.1 by applying **Box–Muller transform** starting from uniform samples generated using `np.random.rand()`. You are *not allowed* to use `np.random.normal`.
- (b) The empirical CDF of X is calculated by generating a large number of samples of X and plotting the cumulative proportion of the unique values taken by these samples.
- (c) You have to write your own function for generating the empirical CDF. You should **NOT** use `matplotlib.pyplot.ecdf` or any other such built-in functions.